Physics-Informed Neural Networks for Mechanics: Reproduction and Targeted Improvements

UE24CS352A Machine Learning · Mini-Project Write-up

Team 16 · Problem Statement 13

Abhay A Kora | PES2UG24AM192 · Anusha B | PES2UG24AM195 · Section: C-AIML

Faculty: Dr.Kokila Paramanandam

Summary. We re-implemented, from scratch, the three physics-informed neural network (PINN) experiments of our assigned paper and matched its results. We then implemented and tested the two improvements the paper suggests but never tries. **Results:** reproduction errors of about 1 %; a hard boundary condition cut the heat error about 220 times; adaptive training points halved the worst-case consolidation error.

1. Problem statement

Assigned paper: Zhang, Chen and Yang, *Data Driven Solutions and Discoveries in Mechanics Using Physics Informed Neural Network* (Stanford CS229). A PINN is a neural network trained to satisfy a physics equation, so it can solve the equation **without a mesh and without labelled data**, and can even discover unknown parameters. The paper tests this on three problems:

#	Problem	Governing equation	Type	What must be found
1	**Soil consolidation (1D)**	$p_t = p_{xx}$, $0 < x < 1$, $0 < t \le 0.5$	Forward	Pore pressure p(x, t)
2	**Heat conduction (2D)**	$T_{xx} + T_{yy} + 1 = 0$ on $[-1, 1]^2$, $T = 0$ on edges	Forward	Temperature T(x, y)
3	**Damped spring-mass**	$u'' + c\,u' + k\,u = 0$, $u(0) = u'(0) = 1$	Inverse	Unknown c and k from 50 observations

Our objectives: (1) reproduce all three results faithfully; (2) implement the two improvements the paper proposes but does not test, a **hard boundary condition** (§4.2) and **adaptive training-point selection** (§5), and measure whether they help.

2. Dataset

- **No external dataset is needed.** PINNs train on collocation points: random locations in the domain where the equation is checked. They carry no labels; the equation itself supervises the network.
- **Training points (fixed seed):** consolidation 500 interior + 375 boundary/initial; heat 1,500 interior + 500 boundary; spring 100 interior + the initial conditions.
- **Observations (spring only):** 50 evenly spaced values of u(t), generated from the exact solution with c = 0.4, k = 4.
- **Reference answers for testing:** exact formulas for consolidation and the spring; for heat, our own finite-difference solver (201 × 201 grid, accurate to about 4×10^{-6}). About 10,000 test points per problem.

3. Approach

PINN in one line. A tanh network maps coordinates to the solution; automatic differentiation gives its exact derivatives, so we can compute how far it is from obeying the equation (the residual). Training minimises $L = w_f L_f + w_b L_b + w_d L_d$: the equation error, the boundary/initial-condition error and the data error. For the spring problem, c and k are trained alongside the network.

1. **Part 1, reproduction.** Every paper setting: networks 5×50, 4×50 and 3×32; Glorot initialisation; fixed loss weights; Adam for 50,000 epochs then L-BFGS (Adam only, 100,000 epochs, for the spring).
2. **Fix 1, hard boundary condition (heat).** Problem: T = 0 on the edges is only a penalty, so errors concentrate there. Fix: output = $(1 - x^2)(1 - y^2) \times$ network, which is exactly 0 on every edge, so the boundary term is no longer needed.
3. **Fix 2, adaptive training points (consolidation).** Problem: 500 random points are too sparse near t = 0, where the solution changes sharply. Fix: residual-based adaptive refinement (RAR, from DeepXDE [3]): every 10,000 epochs, add the 40 of 5,000 candidate points with the largest residual (500 → 700 points).
4. **Control experiment.** Same schedule and same 200 extra points, but chosen at random. It shows whether RAR helps because of where it puts points, or only because there are more of them. All comparisons use the same network, seed and training budget.

4. Implementation

- **Stack:** Python, PyTorch, NumPy, SciPy, Matplotlib; laptop CPU, about 8 to 16 minutes per run.
- **Structure:** src/ holds the shared parts (network, Adam→L-BFGS trainer, reference solutions, plotting). experiments/ has one script per run: consolidation.py, heat.py, spring_inverse.py (Part 1); heat_improved.py, consolidation_rar.py (Part 2, with a --random-add flag for the control).
- **Running:** run_all.py and run_part2.py reproduce everything; --load reopens a saved model in seconds. Each run saves its metrics, training history, model and figures in results/.
- **Reproducibility:** fixed seeds give identical numbers on reruns (verified by running everything twice).

5. Results

Relative L2 error = overall error as a percentage of the solution's size. All numbers are from one final run on one laptop.

Part 1: reproduction	Paper	Ours	Verdict
Consolidation: relative L2 error	"perfect match"	1.29 %	**Reproduced; largest error near the origin, as in the paper**
Heat: relative L2 error	consistent with FEM	0.90 %	**Reproduced; largest error on the boundary, as in the paper**
Spring: identified c, k (true 0.4, 4)	0.401, 3.993	0.411, 3.990	**Reproduced; within 2.9 % and 0.24 %**

Part 2: improvements	Before (paper's method)	Random +200 (control)	After (our fix)
Fix 1 · Heat: relative L2 error	0.90 %	–	**0.0040 % (≈ 220× lower)**
Fix 1 · Heat: boundary error	3.0×10^{-3}	–	**exactly 0**
Fix 2 · Consolidation: relative L2 error	1.29 %	0.84 %	**0.79 %**
Fix 2 · Consolidation: worst-case error (t > 0)	0.069	0.060	**0.032 (halved)**
Fix 2 · Consolidation: equation error at test points	18.7	99.0	**0.11**

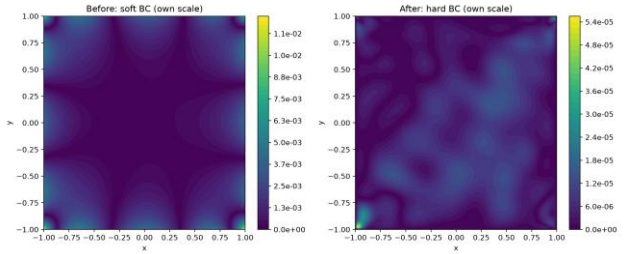


Fig. 1. Heat error before (left, up to 1.2 × 10^{-2}) and after Fix 1 (right, up to 5.5 × 10^{-5}).

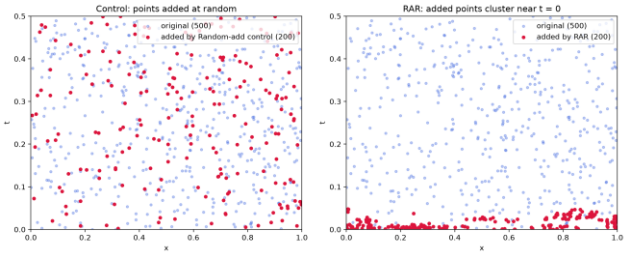


Fig. 2. Where the 200 extra points go: random control (left) vs RAR (right, packed near t = 0).

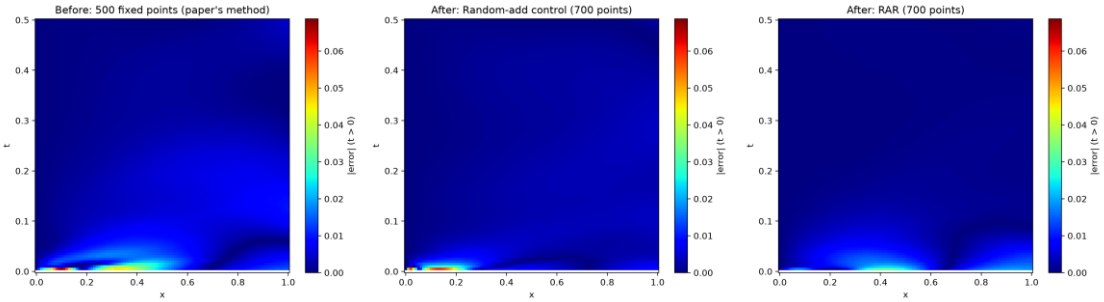


Fig. 3. Consolidation error after t = 0, same colour scale: before, random control, RAR. The bright strip near t = 0 fades with RAR.

What the numbers show:

- **Fix 1** works mainly because the loss now has one term instead of two competing ones: L-BFGS kept improving for 4,019 iterations instead of stalling after 14.
- **Fix 2** halves the worst-case error and removes the large equation error near t = 0. The control shows that the smaller gain in average error comes mostly from adding points, not from where RAR puts them.

6. Conclusions

- A PINN reproduces all three of the paper's results (about 1 % error), and recovers c and k within 3 % from the equation and 50 observations alone.
- How constraints are imposed matters: building the boundary condition into the network cut the heat error about 220 times and made the boundary exact.
- Where training points are placed matters: adaptive refinement fixed the hardest region. The control experiment kept our claim accurate: adaptivity improves the worst case, not the average.
- **Limitations and next steps:** single seed per run (numbers vary slightly across machines); the paper's output limits ($0 \le p \le 1$, $T > 0$) are not described in the paper and were not used. Next: multi-seed averages, a hard initial condition for consolidation, and the coupled poromechanics equations the paper names as future work.

References: [1] Q. Zhang, Y. Chen, Z. Yang, Data Driven Solutions and Discoveries in Mechanics Using PINN, Stanford CS229 report. [2] M. Raissi et al., Physics-informed neural networks, J. Comput. Phys. 378 (2019). [3] L. Lu et al., DeepXDE, arXiv:1907.04502 (2019). [4] J. Berg, K. Nyström, Neurocomputing 317 (2018).